Eficiencia de tokens V2 — Determinismo + Splitting

Fecha: 2026-04-22 · Iteración: V2 (profunda) · Proyecto: Intrale Platform · Pipeline V2

> **Qué cambió respecto del reporte V1 del 2026-04-22 (mañana):** El V1 se enfocó en *prompt caching + TTS gratis*. Este V2 agrega el eje que Leo pidió explícitamente: **cuánto de lo que hoy consume tokens puede hacerse SIN modelo** (determinismo puro) y **cuánto puede bajar de modelo** (Opus → Sonnet → Haiku) vía splitting de agentes en fases.

1. Tesis central

El pipeline hoy trata a casi todos los agentes como "unidad monolítica": se spawnea un proceso `claude`, el rol ya asignado marca un modelo fijo, y el agente hace **todo** con ese modelo. Pero la mayoría de los skills contienen, en realidad, **tres fases muy distintas**:

1. **Preparación** — recolectar inputs: `gh issue view`, `git diff`, lectura de archivos, ejecución de linters, corrida de tests, parseo de reportes XML/JSON. 100% determinístico
2. **Razonamiento** — entender el problema, decidir, generar prosa o código nuevo. requiere LLM pesado
3. **Entrega** — commit, push, crear PR, subir comentarios, actualizar labels, mover cards. 100% determinístico

Hoy, los tres se pagan al mismo precio (modelo del role). Si partimos el flujo en etapas con el modelo correcto, se evita pagar Opus por hacer `gh pr merge`.

2. Inventario de skills — determinismo y modelo óptimo

Skill	Hoy	% determinístico	Modelo óptimo para el resto	Splitting propuesto
`/branch`	Haiku	100%	ninguno	Reemplazar por script Node: leer issue, derivar slug, `git checkout -b`. Cero LLM.
`/monitor`	Haiku	100%	ninguno	Ya está casi todo en `context-cli.js`. Eliminar el skill LLM.
`/cleanup`	Haiku	100%	ninguno	Script Node con reglas de protección (paths Intrale). Ya parcialmente hecho.
`/ops`	Haiku	100%	ninguno	Ya existe `ops-check.js`. El role LLM es un pasamanos.
`/builder`	Haiku	100%	ninguno	Es `./gradlew check` + parseo. Node wrapper ya validado en #2425.
`/cost`	Haiku	100%	ninguno	Fórmula fija sobre `agent-metrics.json`. Un `.js`.
`/perf`	Haiku	100%	ninguno	Parse de `--profile` Gradle. Diff vs baseline. Node puro.
`/priorizar`	Haiku	100%	ninguno	Matriz de labels × reglas. Tabla en JSON. Sin LLM.
`/auth`	Haiku	~90%	Haiku para casos borde	Validación de settings.json + diff: Node. Solo explicación de hallazgo al usuario necesita LLM.
`/delivery`	Haiku	~75%	Haiku solo redacción	git+gh CLI es script. Solo **commit body + PR body** va a LLM (contexto: diff resumido).
`/scrum`	Haiku	~80%	Haiku solo narrativa	Métricas Project V2 = API + cálculo. Solo "insights / standup narrative" va a LLM.
`/doc`, `/historia`,	Haiku	~60%	Haiku	Búsqueda de duplicados + labels + `gh issue create` es

`/refinar`				Node. Redacción va a Haiku con template restrictivo (no necesita Sonnet).
`/guru`	Sonnet	~40%	Haiku para extracción	Context7 fetch + grep en codebase = Node. Solo **viabilidad arquitectónica** requiere Sonnet. El resto (listar dependencias, breaking changes) lo hace Haiku con regex.
`/planner`	Sonnet	~55%	Sonnet solo priorización	Capacity math + carry-over + cerrar sprint = Node. Split heurístico de historias grandes con reglas (archivos tocados, módulos) = Haiku. Solo la priorización del backlog requiere Sonnet.
`/po`	Sonnet	~30%	Sonnet	Chequeo estructural de criterios (¿existen? ¿son testeables?) = Node regex. Solo **juicio de aceptación** requiere Sonnet.
`/ux`	Sonnet	~25%	Sonnet	Chequeo de checklist UX fijo (strings, i18n, accesibilidad por linter) = Node. Solo **crítica de diseño libre** requiere Sonnet.
`/security`	Sonnet	~70%	Haiku para triaje	`npm audit` + `semgrep` + `detect-secrets` ya son determinísticos. Solo el **triaje de falsos positivos** requiere LLM, y Haiku alcanza (la info cruda es estructurada).
`/review`	Opus	~40%	Sonnet mayoría + Opus solo cambios críticos	Chequeos de convenciones (logger declarado, statusCode, Do pattern, `resString`, no stringResource) = **linter ad-hoc**. Solo el análisis semántico libre va a LLM, y mayoritariamente Sonnet es suficiente; Opus solo

				si el diff pasa cierto umbral de complejidad (≥500 LOC o tocar seguridad/auth).
`/tester`	Opus	~65%	Sonnet	Correr `./gradlew check` + parsear Kover XML + diff vs baseline = Node. Solo **proponer tests nuevos** requiere LLM; Sonnet alcanza para ese step.
`/qa`	Opus	~50%	Sonnet + narración edge-tts	Levantar emulador + correr flow script + grabar video = Node (ya está en `qa-android.sh`). Narración TTS = edge-tts (gratis). Solo **análisis del video/logs de fallo** requiere LLM — Sonnet alcanza si los logs vienen estructurados.
`/backend-dev`, `/android-dev`, `/web-dev`, `/pipeline-dev`	Opus	~15%	Opus	Solo la fase de **preparación** (leer issue, checkout branch, scan de archivos tocados) puede hacerse en Node antes de spawnear Opus. Implementación genuinamente requiere Opus. Pero se puede **bajar a Sonnet** si el issue está marcado `complejidad:baja` o `tipo:fix-pequeño`.

Conclusión del inventario: de los 22 skills listados, **8 son 100% determinísticos**, **5 son mayormente determinísticos con un toque de Haiku**, y **solo 4 necesitan genuinamente un modelo grande para el núcleo** (dev/Opus + review complejo).

3. Oportunidades determinísticas fuera de los skills

3.1 Router determinístico en Telegram

> **Situación actual:** el listener (`listener-telegram.js`) encola todo mensaje al commander, que arranca una sesión `claude` para resolver incluso comandos triviales.

Sin embargo, `command-dispatcher.js` ya tiene 13+ handlers Node puros (`/help` , `/status` , `/session` , `/pendientes` , `/retry` , `/limpiar` , `/detalle` , `/restart` , `/reset` , `/dash-*`).

Problema: esos handlers corren *dentro* de una sesión Claude — el proceso se spawneó aunque el resultado no use el modelo. Eso paga carga del system prompt del Commander (~8K tokens) por nada.

Propuesta: agregar un *router determinístico* en `listener-telegram.js` que intercepte comandos reconocidos con regex y los despache a un proceso Node chico (sin spawn de `claude`). Los mensajes que no hacen match van al commander como hoy.

Ahorro: alto — cada `/status` hoy gasta al menos 3-5K tokens solo por arrancar la sesión. Leo tipea `/status` ~8-15 veces/día.

3.2 Compactación del user prompt al spawnear agentes

Hallazgo nuevo crítico. En `pulpo.js:3954` :

```
let userPrompt = `Archivo de trabajo: ...\nContenido:\n${yaml.dump(workData
```

Se inyecta **todo** `workData` sin filtrar. En issues con rebotes, eso incluye histórico completo, logs de builds anteriores, resultados de fases pasadas, comentarios... Fácil 5-20 KB *por invocación*.

Propuesta: pasar solo *paths*, no contenidos. El agente lee lo que necesita con `Read` / `cat` :

```
workDataSlim = {
  issue: workData.issue,
  fase: workData.fase,
  pipeline: workData.pipeline,
  rebote: workData.rebote,
  rechazado_en_fase: workData.rechazado_en_fase,
  motivo_rechazo: truncate(workData.motivo_rechazo, 500),
  evidence_paths: {
    build_log: `.pipeline/logs/build-${issue}.log`,
    previous_phase: `.pipeline/procesado/${issue}.*`,
    rebote_history: `.pipeline/rebotes/${issue}.jsonl`
```

```
  }
}
```

Ahorro estimado: ~3-7 KB por spawn × ~200 spawns/semana ≈ **50-100K tokens/semana de input**, sin perder información (el agente puede leer bajo demanda).

3.3 Templates de rejection report (ya es determinístico — verificado)

`rejection-report.js:1396-1499` ya es 100% template Node. No hay LLM generando la narración, solo interpolación de strings. **OK, no tocar.** (El V1 sugería mejoras marginales; vale la pena solo si se hace el refactor de templates globales.)

3.4 Prompt caching (sigue siendo la mayor ganancia absoluta)

Confirmado del V1: el rol base + specific suma 3.5-14.5 KB por spawn, sin `cache_control`. **Ahorro estimado: 40-70% en input tokens**. Nada cambió.

3.5 Heartbeats del pulpo

`pulpo.js:4866-4898` (`generarMensajeProgreso`): **ya es template Node**, sin LLM. **OK.**

3.6 Hooks

El análisis del hook system mostró solo 2 hooks activos (`PreToolUse: branch-guard.js` , `PostToolUse: activity-logger.js`). Los "68 hooks" mencionados en V1 son archivos `.js` en disco, pero no todos registrados — falso positivo parcial. **Hook overhead real es bajo.**

4. Splitting concreto — 4 casos prioritarios

4.1 `/review` — el caso más grande

Antes: Opus hace todo: leer PR, diff, evaluar convenciones, análisis semántico, escribir comentarios.

Propuesta (3 fases):

1. Node `review-lint.js` : valida convenciones Intrale con regex (logger declarado, statusCode en response, patrón Do, `resString` , no `stringResource` fuera de `ResStrings`). Output: JSON de hallazgos.
2. Sonnet análisis semántico del diff filtrado (solo archivos con cambios no triviales, excluyendo import reorders y format-only). Lee el JSON de (1) como contexto estructurado.
3. Opus solo si: `diff_loc > 500` OR `touches_auth` OR `touches_payment` OR label `risk:high` .
4. Node post-proceso: publicar comentarios en GH vía `gh api` , bajar checks, etiquetar.

Ahorro esperado: ~60% del costo actual de `/review` .

4.2 `/qa` — caso multi-etapa

Propuesta:

1. Node `qa-android.sh` / `qa-api.sh` : levantar emulador, instalar APK, correr flow scripted, grabar video, guardar logcat. (Ya existe.)
2. Node parsear logcat buscando `FATAL EXCEPTION` , `ANR` , `CRASH` . Si no hay → **pass automático** (no se gasta un solo token).
3. Sonnet solo si hay fallo: recibir el fragmento relevante del logcat + transcripción simple del video (frames clave con OCR, que es determinístico), decidir causa.
4. Node edge-tts genera narración del veredicto con template.

Ahorro esperado: en runs sin fallo (la mayoría) → **cero LLM**. Hoy siempre se ejecuta Opus.

4.3 `/tester` — chequeos con baseline

Propuesta:

1. Node correr `./gradlew check` , parsear Kover XML, comparar cobertura vs baseline (`kover-baseline.json`).
2. Si tests pasan + cobertura ≥ baseline → **aprobar sin LLM**.

3. Sonnet solo si hay gap: leer el diff + reportes, proponer 2-5 casos nuevos en template.

Ahorro: ~70% de runs (cuando ya hay tests suficientes) saltan el LLM.

4.4 `/security` — bajar a Haiku

Propuesta:

1. Node `npm audit`, `semgrep`, `detect-secrets`, `gradle dependencyCheck`. Output: JSON estructurado de hallazgos.
2. Si 0 hallazgos críticos/altos → **aprobar sin LLM**.
3. Haiku para triaje de falsos positivos (la info cruda ya está estructurada, no requiere razonamiento complejo).
4. Node abrir issues de seguridad con template por cada hallazgo confirmado.

Ahorro: pasar de Sonnet a Haiku + salto completo cuando no hay hallazgos → **~80% del costo de /security**.

5. Roadmap consolidado (V2)

Fase	Ítem	Esfuerzo	Ahorro	Precondición
F0 · Ya	P1.3 V1: tracking real de tokens	Simple	bloqueante para medir	ninguna
F1 · Determinismo barato	Router Telegram (§3.1)	Simple	-15-25% tokens Commander	F0
F1	Compactar workData (§3.2)	Simple	-50-100K tokens/semana input	F0
F1	Prompt caching (V1 §P1.1)	Simple	-40-70% input repetido	F0
F1	Edge-TTS primario (V1 §P1.2)	Simple	-100% TTS OpenAI	ninguna

F2 · Migración de skills a Node	Eliminar LLM de /branch, /monitor, /cleanup, /ops, /cost, /perf, /priorizar	Medio	-100% costo de esos skills	F1 tracking
F2	Migrar /builder a Node wrapper	Simple	-100% LLM (gradle ya corre)	ninguna
F3 · Splitting	/security bajado a Haiku + salto en 0 hallazgos (§4.4)	Medio	-80% /security	F0, F2
F3	/tester con baseline-skip (§4.3)	Medio	-70% runs sin cambios	F0
F3	/qa multi-etapa con pass-auto (§4.2)	Grande	-90% costos QA cuando no hay fallo	F0, OCR de video
F3	/review con linter + ruteo por complejidad (§4.1)	Grande	-60% costo review	F0, reglas en linter
F4 · Avanzado	Sesiones persistentes en rebotes (V1 §P3.2)	Grande	-70-90% en re-intentos	F1 caching
F4	Budget duro de tokens por skill (V1 §P2.3)	Medio	proteje runaways	F0

6. Target combinado realista

-60% Anthropic total en 3 meses · **-100% OpenAI TTS**

-30-40% de lo actual solo con F1 (1-2 semanas)
+20-30% adicional con F2 (2-3 semanas)
+15-20% adicional con F3 (1 mes)

7. Principios de diseño (para no volver a caer en lo mismo)

1. **LLM es para decisiones, no para movimiento de datos.** Si una tarea se puede describir como "leer X, aplicar regla Y, escribir Z", no debería tener LLM.
2. **Cuanto más estructurado el input, más barato el modelo que alcanza.** Dar a Sonnet un JSON de hallazgos > dar a Opus el log crudo.
3. **El agente no debe recibir lo que puede leer él mismo.** Paths > contenidos. Herramientas > dumps.
4. **Pass-through automático cuando no hay señal.** Tests verdes + cobertura OK + sin hallazgos = aprobar sin consultar LLM.
5. **Splitting por fase, no por feature.** Preparación y entrega son Node; razonamiento es LLM.

8. Issues a abrir (propuesta)

#	Título	Prioridad	Labels
1	[pipeline] Tracking real de tokens desde stream-json del CLI claude	high	area:pipeline, tipo:infra
2	[pipeline] Router determinístico en listener-telegram.js para comandos Node-only	high	area:pipeline, tipo:infra
3	[pipeline] Compactar workData en user prompt — pasar paths en vez de contenidos	high	area:pipeline, tipo:infra
4	[pipeline] Activar prompt caching en spawn de agentes	high	area:pipeline, tipo:infra
5	[pipeline] Edge-TTS como primario, OpenAI como fallback	medium	area:pipeline, tipo:infra
6	[pipeline] Migrar skills determinísticos a Node puro (/branch, /monitor, /cleanup, /ops, /cost, /perf, /priorizar)	medium	area:pipeline, tipo:infra
7	[pipeline] /security con ejecución determinística + salto cuando 0 hallazgos	medium	area:pipeline, tipo:infra
8	[pipeline] /tester con baseline-skip cuando tests pasan y cobertura no baja	medium	area:pipeline, tipo:infra
9	[pipeline] /qa multi-etapa: pass-auto cuando logcat no muestra crash	medium	area:pipeline, tipo:infra

10	[pipeline] /review con linter ad-hoc + ruteo por complejidad (Sonnet default, Opus solo si risk:high)	medium	area:pipeline, tipo:infra
11	[pipeline] Budget duro de tokens por skill (cortar runaways)	low	area:pipeline, tipo:infra
12	[pipeline] Sesiones persistentes para rebotes (reuso de contexto)	low	area:pipeline, tipo:infra

---

Generado 2026-04-22 · Intrale Platform Pipeline V2 · Iteración V2 del reporte de eficiencia